L0 Cyclic-resource control

This example set contains 2 instances for the Level 0 problem curriculum:

- minimal_manufacturing_cell is a small teaching model. Its purpose is to make the control problem and the unsafe marking easy to understand.
- flexible_manufacturing_cell adds product variety, alternative routes, overlapping resource cycles and a three-job release constraint.

Document visibility

This document contains artefacts for 3 readers:

Sections in this document	Final Exam artefact	Who sees it
Domain Context, Dataset	Phase A input	Phase A contestant, and everyone below
Model, KPIs, Specification, Baseline policies	Reference solution ($\mathcal{M}$, $\mathcal{K}$, φ, π_o)	Phase A examiner; Phase B contestant
Supplementary material	N/A (not referenced in Final exam)	Examiners only (and Final exam authors)

Model 1: minimal cyclic resource cell

This model exists primarily to explain the phenomenon.

Complexity dimensions

Dimension	Classification	Rationale
Semantic expressivity	L0	Uncoloured, untimed Petri net with standard arcs and fixed firing effects.
Node size	Small	12 places and 6 transitions.
Time horizon	Not applicable	The net is untimed.
Marking graph and state space	Finite	There are three order tokens and one token for each reusable machine.
Modularity	Monolithic	The complete manufacturing cell is defined in one net.

Dimension	Classification	Rationale
Determinism	Non-deterministic run	Every transition has a deterministic effect, but several transitions can be enabled at once.
Stochastic process class	Not applicable	There are no stochastic delays, continuous dynamics or diffusion.
Verification objective	Safety and guarantee	A policy must avoid unfinished deadlock while completing every order at the end of every maximal policy-constrained run.

Domain Context

A manufacturing cell handles products which need two shared machines. When a job starts, it reserves its first machine and retains that machine until its second operation can be performed. The cell has 3 machines, and each machine is shared by 2 products:

- Machine A: first machine of A-to-B, second machine of C-to-A.
- Machine B: first machine of B-to-C, second machine of A-to-B.
- Machine C: first machine of C-to-A, second machine of B-to-C.

When a job starts, it takes its first machine and holds it. It releases that machine only after it finishes on its second machine. A job cannot start its second operation until that machine is free.

If all three jobs start before any finishes, each holds one machine and waits for the machine held by the next job. Once this circular wait is formed, no job can be completed.

The decision problem is to find a dispatch policy which:

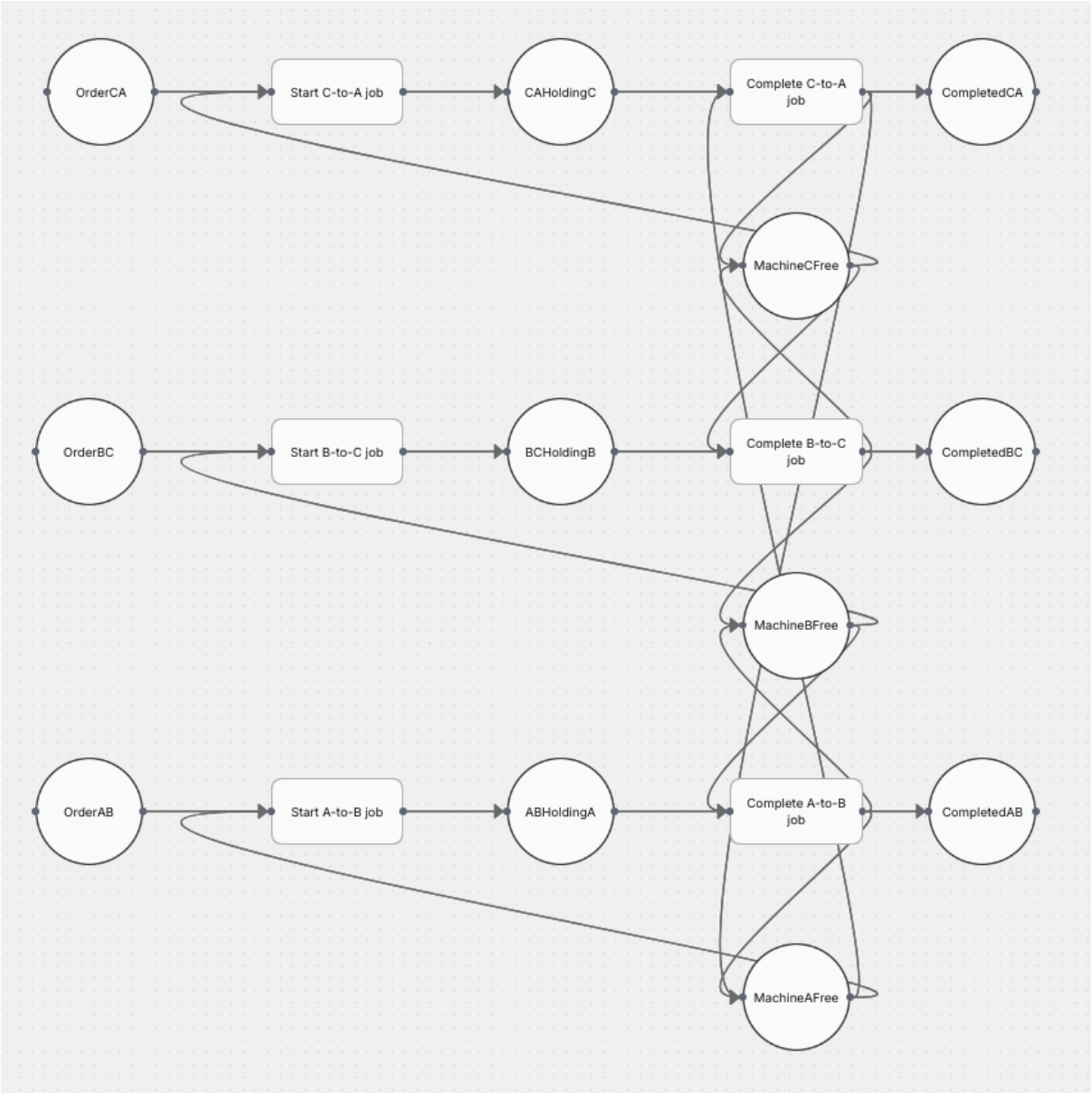
1. prevents unfinished deadlocks
2. completes all orders rather than obtaining safety by blocking production

Dataset

There are no datasets associated with this instance.

Model

The model file can be found here.



KPIs

The below KPIs define the marking as a function $M: P \to \mathbb{N}$, where $M(p)$ is the number of tokens in place p.

Completed orders

Total number of completed jobs across all order types:

$$k_{done}(M) = \sum_{p \in P_{done}} M(p)$$

Where:

$P_{done} = \{CompletedAB, CompletedBC, CompletedCA\}$

Open orders

Total number of open orders in waiting and work-in-progress places:

$$k_{open}(M) = \sum_{p \in P_{wait} \cup P_{wip}} M(p)$$

Where:

$P_{wait} = \{OrderAB, OrderBC, OrderCA\}$

$P_{wip} = \{ABHoldingA, BCHoldingB, CAHoldingC\}$

Deadlock

Deadlock state equals to 1 when open orders remain but no transitions can fire, 0 otherwise:

$$\mathrm{Dead}(M) = \begin{cases} 1 & \text{if } k_{\mathrm{open}}(M) > 0 \text{ and } \sum_{p \in P_{\mathrm{free}}} M(p) = 0 \\ 0 & \text{otherwise} \end{cases}$$

Where $P_{free} = \{MachineAFree, MachineBFree, MachineCFree\}$

Note, in this instance every transition needs a free machine and any free machine can always be used by some waiting job, so "no free machine" means no transitions can fire.

Specification

Each specification holds if the probability that a single run meets its condition is at least $1 - \delta$, where δ is pre-registered per specification. $\delta = 0$ for both requirements:

φ1 Deadlock freedom (safety):

$\mathbb{P}[Dead(M) = 0] \geq 1 - \delta,\ \delta = 0$ at every reachable marking under the policy

φ2 Completion:

$\mathbb{P}\left[k_{done}(M) = 0\right] \geq 1 - \delta,\ \delta = 0$ at the end of every maximal run under the policy

φ3 Fairness:

- $\mathbb{P}[Every\ time\ (M(OrderAB) > 0) \rightarrow eventually\ (M(CompletedAB) > 0)] \geq 1 - \delta,\ \delta = 0$
- $\mathbb{P}[Every\ time\ (M(OrderBC) > 0) \rightarrow eventually\ (M(CompletedBC) > 0)] \geq 1 - \delta,\ \delta = 0$
- $\mathbb{P}[Every\ time\ (M(OrderCA) > 0) \rightarrow eventually\ (M(CompletedCA) > 0)] \geq 1 - \delta,\ \delta = 0$

where p → q means "if p happens, then q happens". Satisfying this specification guarantees no single job (AB, BC, CA) can be postponed forever while other jobs keep getting serviced.

Baseline policies & levers

Policy levers

An AI controller can act on 3 transitions to start a job. Dispatching a job takes it out of the queue and puts it on its first machine, which it holds until the required second machine is free.

Transition	Enabled when	Effect when fired
Start A-to-B job	OrderAB ≥ 1 and MachineAFree = 1	Job holds Machine A and waits for Machine B
Start B-to-C job	OrderBC ≥ 1 and MachineBFree = 1	Job holds Machine B and waits for Machine C
Start C-to-A job	OrderCA ≥ 1 and MachineCFree = 1	Job holds Machine C and waits for Machine A

Baseline policies π_0

The model's default baseline policy includes no control logic, and in effect implements:

- **Greedy utilisation:** dispatches a waiting job whenever its first machine is free.

The greedy-utilisation baseline can produce the three-job circular wait.

Alternative policies

Other potential policies that an AI controller may implement are:

- **Serial dispatch:** admits one job and permits it to complete before admitting another.
- **Random dispatch:** chooses uniformly from enabled transitions (same as greedy utilisation but with randomisation).

Model 2: flexible manufacturing cell

The second model builds on the minimal instance to a more meaningful policy-learning problem. A policy must select three initial dispatches before any processing can complete, so serialising the entire workload is not available as a trivial solution. It must reason about the global wait-for structure and choose between alternative routes.

Complexity dimensions

Dimension	**Classification**	**Rationale**
Semantic expressivity	L0	Uncoloured, untimed Petri net with standard arcs and fixed firing effects.
Node size	Small	29 places and 19 transitions, with 6 product families and 5 shared machines.
Time horizon	Not applicable	The net is untimed. A training harness may impose a transition-step cutoff, but this must not be interpreted as physical time.
Marking graph and state space	Finite	The initial order and machines bound the number of tokens relevant to production.
Modularity	Monolithic	The complete manufacturing cell is defined in one net.
Determinism	Non-deterministic run	Transition effects are deterministic, while route conflicts and firing order produce non-deterministic executions.
Stochastic process class	Not applicable	There are no stochastic delays, continuous dynamics or diffusion.
Verification objective	Safety and guarantee	A policy must avoid unfinished deadlock and complete every order, with an optional finite-episode performance objective among safe policies.

Domain context

A manufacturing cell has five machines: cutter, mill, paint booth, test rig and packing cell. Six product types are produced. Each product needs two machines in sequence.

The product holds its first machine until it finishes on its second machine. The products and their routes are listed below:

Product	Standard route	Alternative route
Frame	cutter → mill	—
Panel	mill → paint booth	test rig → paint booth
Cabinet	paint booth → cutter	packing cell → cutter
Sensor	paint booth → test rig	—
Circuit board	test rig → packing cell	—
Pump	packing cell → paint booth	mill → paint booth

The production cell operates in release waves. Three jobs are released at once. No job can finish its second operation until all three have been released. Each time a job completes, one new release becomes available. At most three jobs occupy machines at any time.

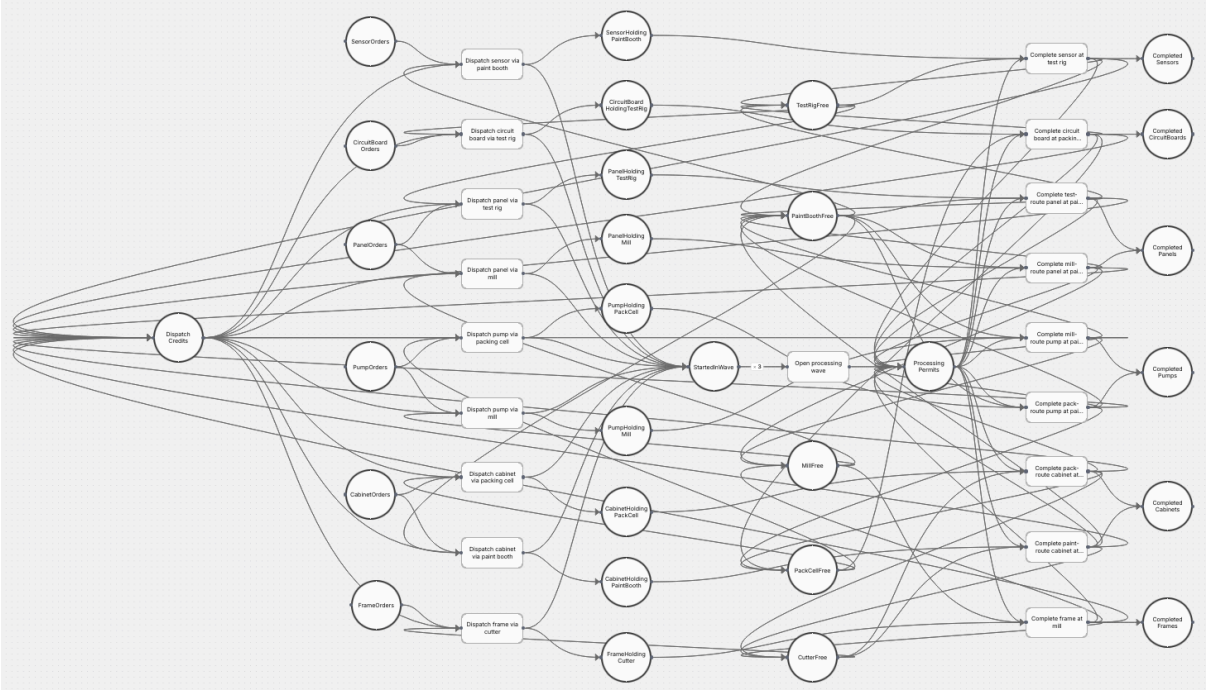
The operator chooses which product to release and, for panels, cabinets and pumps, which of the two routes to assign. Processing and completion happen automatically once the required machines are free. The release strategy should ensure no circular wait forms and every order completes.

Dataset

There are no datasets associated with this instance.

Model

The model file can be found here.



KPIs

The below KPIs define the marking as a function $M: P \to \mathbb{N}$, where $M(p)$is the number of tokens in place p.

Completed orders

Total number of completed jobs across all order types:

$$k_{open}(M) = \sum_{p \epsilon P_{done}} M(p)$$

Where:

$$P_{done} = \{CompletedFrames, CompletedPanels, CompletedCabinets, \\ CompletedSensors, CompletedCircuitBoards, CompletedPumps\}$$

Weighted completion score

The points earned by orders completed according to product type, where valuable orders score higher.

$$k_{open}(M_s) = \sum_{p \epsilon P_{done}} w_p M_s(p)$$

Where:

$w = \{w_{Frames}, w_{Panels}, w_{Cabinets}, w_{Sensors}, w_{Circuit\ boards}, w_{Pumps}\} = (2, 3, 4, 5, 6, 5)$

Open orders

Total number of open orders either waiting to be dispatched or holding their first machine while waiting for the second.

$$k_{open}(M) = \sum_{p \epsilon P_{wait} \cup P_{wip}} M(p)$$

Where:

$P_{wait} = \{FrameOrders, PanelOrders, CabinetOrders, SensorOrders, CircuitBoardOrders, PumpOrders\}$

$P_{wip} = \{FrameHoldingCutter, PanelHoldingMill, PanelHoldingTestRig, CabinetHoldingPaintBooth, CabinetHoldingPackCell, SensorHoldingPaintBooth, CircuitBoardHoldingTestRig, PumpHoldingPackCell, PumpHoldingMill\}$

Machines in use

The number of 5 machines currently held by a job:

$$k_{busy}(M) = 5 - \sum_{p \epsilon P_{free}} M(p)$$

Where:

$P_{done} = \{CutterFree, MillFree, PaintBoothFree, TestRigFree, PackCellFree\}$

Unfinished deadlock

Equals to 1 when open orders remain but no transitions can fire, 0 otherwise:

$$\mathrm{Dead}(M) = \begin{cases} 1 & \text{if } k_{\mathrm{open}}(M) > 0 \text{ and } \{\, t \in T : \mathrm{en}(M, t) \,\} = \emptyset \\ 0 & \text{otherwise} \end{cases}$$

Where $en(M, t)$ = transition t enabled at marking M

Specification

Each specification holds if the probability that a single run meets its condition is at least $1 - \delta$, where δ is pre-registered per specification. $\delta = 0$ for both requirements:

φ1 Deadlock safety:

$\mathbb{P}[Dead(M) = 0] \geq 1 - \delta,\ \delta = 0$ at every reachable marking under the policy

φ2 Completion:

$\mathbb{P}\left[k_{open}(M) = 0\right] \geq 1 - \delta,\ \delta = 0$ at the end of every maximal run under the policy

Baseline policies & levers

Policy levers

The intended AI controller has to make two decisions at each release:

- Which product to release. Any product family with waiting orders and whose first machine is free.
- Which route to assign (panels, cabinets and pumps only). The choice determines which machine the product occupies first.

On the model, the permissible actions the controller can take are:

Controllable transition	**Product**	**Route**
Dispatch frame via cutter	Frame	standard (only)
Dispatch panel via mill	Panel	standard
Dispatch panel via test rig	Panel	alternative
Dispatch cabinet via paint booth	Cabinet	standard
Dispatch cabinet via packing cell	Cabinet	alternative
Dispatch sensor via paint booth	Sensor	standard (only)
Dispatch circuit board via test rig	Circuit board	standard (only)
Dispatch pump via packing cell	Pump	standard
Dispatch pump via mill	Pump	alternative

The various product routes create circular dependencies among machines. Three products whose routes form a chain can hold each other's machines:

- cutter → mill → **paint booth** → cutter (used by frames, standard-route panels and standard-route cabinets).
- **paint booth** → test rig → packing cell → paint booth (used by sensors, controllers and standard-route pumps).
- **paint booth** ↔ test rig. (used by senor and alternative-route panel)

The routes overlap in paint booth: a job from any cycle holding the paint booth can block a job from any other cycle that needs it. Whether a dispatch is safe depends on what every machine in the cell is doing, not just whether the target machine is free. A rule that checks one machine at a time, such as "dispatch whenever a machine is free", cannot distinguish safe dispatches from unsafe ones.

The controller cannot change a product's route after release, reorder jobs at a machine, or delay / speed up processing. The controller cannot release jobs one at a time as the release-wave rule forces three releases before any job can finish. After that, each completion unlocks one more release. Completions and wave openings happen automatically.

The metadata attached to transitions in the model defines the intended control boundary:

- control: "controllable" identifies dispatch and route-selection actions.
- control: "uncontrollable" identifies plant completion and release-wave transitions.

At each reachable marking, the policy should return the controllable transitions it permits. Uncontrollable transitions remain permitted.

Baseline policies π_0

The model’s default baseline policy includes no control logic, and in effect implements:

- **Greedy utilisation:** dispatches whenever a credit and first machine are available.

Alternative policies

Other potential policies that an AI controller may implement are:

- **Random dispatch:** chooses any enabled controllable transition (same as greedy utilisation but with randomisation).
- **Fixed standard routes:** only use the default route for panels, cabinets and pumps. Drops 3 of the 9 release options.
- **One-step deadlock avoidance:** blocks a release if the resulting marking itself is a deadlock.

- **Exact winning-set policy:** blocks a release if full completion is no longer guaranteed from the resulting state. Computed by reachability analysis.

Supplementary materials

Model 1: minimal cyclic resource cell

How the model works

There are three jobs:

- OrderAB needs Machine A followed by Machine B.
- OrderBC needs Machine B followed by Machine C.
- OrderCA needs Machine C followed by Machine A.

Each dispatch transition consumes a waiting order and its first free-machine token, then produces a work-in-progress token. Each completion transition consumes that work-in-progress token and the second free-machine token, produces a completed job, and returns both machine tokens.

The unsafe trace is:

1. Fire Start A-to-B job. The A-to-B job holds Machine A.
2. Fire Start B-to-C job. The B-to-C job holds Machine B.
3. Fire Start C-to-A job. The C-to-A job holds Machine C.

No completion transition is then enabled:

- A-to-B waits for B;
- B-to-C waits for C; and
- C-to-A waits for A.

A safe trace completes at least one admitted job before dispatching the job which would close the cycle. For example:

1. start A-to-B;
2. complete A-to-B;
3. start B-to-C;
4. complete B-to-C;
5. start C-to-A; and

6. complete C-to-A.

Scenarios and experiments

Scenario	Description
One job of each route	One A-to-B, one B-to-C and one C-to-A order and a free machine per type (A, B, C)

Experiments worth running:

- **Deadlock:** run the one job of each scenario to check if the baseline policy creates deadlocks.

Place and transition summary tables

Transition	Domain area
Start A-to-B job	Dispatch
Complete A-to-B job	Process completion
Start B-to-C job	Dispatch
Complete B-to-C job	Process completion
Start C-to-A job	Dispatch
Complete C-to-A job	Process completion

Place	Holds	Waits for	Domain area
OrderAB	—	—	Order queue
OrderBC	—	—	Order queue
OrderCA	—	—	Order queue
MachineAFree	—	—	Shared resource
MachineBFree	—	—	Shared resource
MachineCFree	—	—	Shared resource
ABHoldingA	Machine A	Machine B	Work in progress
BCHoldingB	Machine B	Machine C	Work in progress

Place	Holds	Waits for	Domain area
CAHoldingC	Machine C	Machine A	Work in progress
CompletedAB	—	—	Completed output
CompletedBC	—	—	Completed output
CompletedCA	—	—	Completed output

Model 2: flexible manufacturing cell

How the model works

Order queues

Each product family has a queue of waiting orders (FrameOrders, PanelOrders, CabinetOrders, SensorOrders, CircuitBoardOrders, PumpOrders). Orders wait here until the controller releases them.

Dispatch and route

A release consumes one order from the queue, one dispatch credit and the first machine token, then places the job in work in progress. The 3 product routes are outlined in the table.

Shared resources

Each machine has one token (CutterFree, MillFree, PaintBoothFree, TestRigFree, PackCellFree). A dispatch removes the token. A completion returns both machines' tokens.

Release control

DispatchCredits starts at three. Each dispatch consumes one credit and adds one token to StartedInWave. When StartedInWave reaches three, Open processing wave fires automatically: it consumes all three and creates one ProcessingPermits token.

Each completion consumes and immediately returns the permit, and also returns one dispatch credit. So the permit is never spent to enable completions permanently. The credit returns to the pool, making one new dispatch available.

Work in progress

Each dispatched job occupies a holding place (e.g. FrameHoldingCutter, PanelHoldingMill, SensorHoldingPaintBooth). The job stays here, holding its first machine, until its second machine is free.

Process completion

When a job's second machine is free and a processing permit is present, the completion transition fires automatically. It releases both machines, returns one dispatch credit and moves the job to its completed-output place.

Completed output

Each product family has a completion place (CompletedFrames, CompletedPanels, etc.). A successful run ends with all orders here and all other places empty.

Resource cycles

Because jobs from different release waves coexist, a circular wait can form at any dispatch not just in the first wave. Of 1,209 deadlocked states in the balanced scenario, only 6 are reachable after exactly 3 dispatches. The majority occur after 7–8 dispatches, when the cell has processed several completions and started replacement jobs that happen to close a resource cycle with the remaining work in progress.

One unsafe first wave is:

- frame via cutter
- panel via mill
- cabinet via paint booth

After the wave opens, those three jobs wait on the mill, paint booth and cutter respectively. All three machines in the first cycle are held, no credit remains, and no completion is possible.

One safe first wave is:

- frame via cutter
- sensor via paint booth
- cabinet via packing cell

The frame can acquire the free mill and the sensor can acquire the free test rig, so at least one completion is possible. But a safe first wave does not make the rest of the run safe. The controller must check every subsequent dispatch against the jobs currently in progress.

Scenarios and experiments

Scenario	Description
Balanced workload	Two orders from every product family (12 total) and one free machine per type
Shared-stage pressure	18 orders with more on routes using the mill, paint booth and packing cell.
Dispatch credits with one order per product	one order from every product family (6 total) and one free machine per type, with dispatch credits are a parameter that can be varied

Experiments worth running:

- **Deadlocks:** run both scenarios to check if the baseline policy creates deadlocks.
- **Dispatch credits:** sweep the dispatch credits to see how it impacts deadlocks.

Place & Transition Summary tables

Transition	Domain area
Dispatch frame via cutter	Dispatch and route
Complete frame at mill	Process completion
Dispatch panel via mill	Dispatch and route
Complete mill-route panel at paint booth	Process completion
Dispatch panel via test rig	Dispatch and route
Complete test-route panel at paint booth	Process completion
Dispatch cabinet via paint booth	Dispatch and route
Complete paint-route cabinet at cutter	Process completion
Dispatch cabinet via packing cell	Dispatch and route
Complete pack-route cabinet at cutter	Process completion
Dispatch sensor via paint booth	Dispatch and route
Complete sensor at test rig	Process completion
Dispatch circuit board via test rig	Dispatch and route
Complete circuit board at packing cell	Process completion
Dispatch pump via packing cell	Dispatch and route
Complete pack-route pump at paint booth	Process completion
Dispatch pump via mill	Dispatch and route
Complete mill-route pump at paint booth	Process completion
Open processing wave	Batch release

Place	Holds	Waits for	Domain area
FrameOrders	—	—	Order queue
PanelOrders	—	—	Order queue
CabinetOrders	—	—	Order queue
SensorOrders	—	—	Order queue
CircuitBoardOrders	—	—	Order queue

Place	Holds	Waits for	Domain area
PumpOrders	—	—	Order queue
CutterFree	—	—	Shared resource
MillFree	—	—	Shared resource
PaintBoothFree	—	—	Shared resource
TestRigFree	—	—	Shared resource
PackCellFree	—	—	Shared resource
DispatchCredits	—	—	Release control
StartedInWave	—	—	Release bookkeeping
ProcessingPermits	—	—	Release control
FrameHoldingCutter	Cutter	Mill	Work in progress
PanelHoldingMill	Mill	Paint booth	Work in progress
PanelHoldingTestRig	Test rig	Paint booth	Work in progress
CabinetHoldingPaintBooth	Paint booth	Cutter	Work in progress
CabinetHoldingPackCell	Packing cell	Cutter	Work in progress
SensorHoldingPaintBooth	Paint booth	Test rig	Work in progress
CircuitBoardHoldingTestRig	Test rig	Packing cell	Work in progress
PumpHoldingPackCell	Packing cell	Paint booth	Work in progress
PumpHoldingMill	Mill	Paint booth	Work in progress
CompletedFrames	—	—	Completed output
CompletedPanels	—	—	Completed output
CompletedCabinets	—	—	Completed output
CompletedSensors	—	—	Completed output
CompletedCircuitBoards	—	—	Completed output
CompletedPumps	—	—	Completed output